Wiki - non-strict: https://wiki.haskell.org/Non-strict_semantics
Wiki - strictness: https://wiki.haskell.org/Performance/Strictness
Wiki - lazy vs non-strict: https://wiki.haskell.org/index.php?title=Lazy_vs._non-strict

strictness (denotational semantics)
strict semantics
Non-strict semantics
A function is defined to be **strict**

f is **strict** <=> **f undefined = undefined**

A language in which all functions are strict is said to have **strict semantics**. Otherwise, it has **non-strict semantics**.
e.g.
strict language most imperative languages JavaScript, Python, etc
non-strict language Haskell

Thus, in non-strict semantics, it possible to write an expression with an **invalid expressions**
i.e. expressions with a **bottom.**
i.e. an expression with invalid subexpressions can still **evaluate** to a valid value as long as it does not reach the invalid point.

e.g. types of invalid expressions

```
-- ignored errors
const 1 undefined           -- 1
head [1,undefined]          -- 1

-- infinite loop
x = 1:x                     -- [1,1,...]
x                           -- ERROR
take 2 x                    -- OK [1,1]

-- infinite list
[1..]                       -- ERROR. [1,1,...]
take 2 [1..]                -- OK. [1,1]
```

***) strictness in Haskell**
Since Haskell has non-strict semantics, most expressions are **non-strict by default**.
e.g. only **some** functions are strict
e.g. data types are **non-strict**

a) strictness in **functions**
As above, definition of strictness is:

f is **strict** <=> **f undefined = undefined**

For functions of multiple arguments
e.g. if f :: a -> b -> c

f is **strict in a** <=> **f undefined x = undefined**

e.g.

```
f :: a -> a                    -- strict
f x = x
f undefined = undefined

f :: a -> b -> a               -- strict in a
f x y = x
f undefined 0 = undefined
f 0 undefined = 0

f :: a -> Bool                 -- non-strict
f x = True
f undefined = True

f :: a -> a                    -- non-strict
f x = (x, x)
f undefined = (undefined, undefined)

f :: Int -> a                  -- strict since `x > 0` is
strict
f x = if x > 0 then ...
f undefined = undefined
```

The implementation of Functional Programming Languages (1987)

2.5.4 Strictness and Laziness

We say that a function is *strict* if it is sure to need the value of its argument. This is a concept that will arise repeatedly in the book. Can we give a denotational definition of strictness?

If a function, f, is sure to need the value of its argument, and the evaluation of the argument will not terminate, then the application of f to the argument will certainly fail to terminate. This verbose, operational argument suggests the following concise, denotational, definition of strictness:

DEFINITION

A function f is *strict* if and only if

$f \perp = \perp$

The definition generalizes easily to functions of several arguments. For example, if g is a function of three arguments, then g is strict in its second argument if and only if

$g\ a\ \perp\ c = \perp$

for all values of a and c.

If a function is non-strict, we say that it is *lazy*. Technically, this is an abuse of terminology, since lazy evaluation is an implementation technique which implements non-strict semantics. However, 'lazy' is such an evocative term that it is often used where 'non-strict' would be more correct.

seq

seq: https://www.haskell.org/onlinereport/haskell2010/haskellch6.html#x13-1260006.2
Bang pattern: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/strict.html

seq enforces **strictness** upto **weak head normal form (WHNF).**

```
undefined `seq` ...   = undefined
... `seq` undefined   = undefined

- upto WHNF
undefined        :: Maybe a `seq` () = undefined
(Just undefined) :: Maybe a `seq` () = ()
```

Without **seq**, it is impossible to always ensure strictness

```
{- LANGUAGE BangPattern -}

f :: a -> b          -- strict only if g is also strict
f a = g ...

f :: a -> b          -- always strict regardless of g
f a = a `seq` g ...
f !a = g ...         -- same with !
```

e.g.

```
{- LANGUAGE BangPattern -}

f :: a -> (a, a)                          -- non-strict
f x = (x, x)

f' :: a -> (a, a)                         -- strict
f' x = x `seq` (x, x)
f' !x = (x, x)                            -- same
with !

-- e.g.
f undefined = (undefined, undefined)      -- non-strict
f' undefined = undefined                  -- strict
```

b) strictness in **data types**
Strictness flags: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-620004

data type is **non-strict** by **default**.
newtype is **strict** by **default**.
A type can be made **strictness flag !**

e.g.

```
data A a = A a                    -- non-strict by
default
data A' a b = A' !a b             -- strict with
strictness flag
newtype B b = B b                 -- strict by default

-- e.g.
A undefined = A undefined         -- non-strict
A' undefined 1 = undefined        -- strict in a
A' 1 undefined = A' 1 undefined   -- non-strict in b
B undefined = undefined
```

spine strict vs value strict
Types of strictness in a **container** type of data.
A container is **spine strict** if it is strict in its **container**.
A container is **value strict** if it is strict in its **element**.

e.g. list

```
data List a = Cons a (List a) | Nil            --
neither
Cons undefined undefined                       -- OK

data List a = Cons a !(List a) | Nil           -- spine
strict only
Cons undefined undefined                       -- BAD
undefined
Cons undefined Nil                             -- OK.
```

```
value may be undefined


data List a = Cons !a (List a) | Nil           -- value
strict only
Cons undefined undefined                       -- BAD
undefined
Cons undefined Nil                             -- OK.
spine may be undefined

data List a = Cons !a !(List a) | Nil          -- both
spine and value strict
Cons undefined Nil                             -- BAD
undefined
Cons () undefined                              -- BAD
undefined
```

e.g. Data.Map
https://hackage-content.haskell.org/package/docs/src/Data.Map.Internal.html#Map

Data.Map is **spine strict** for both Data.Map.Lazy and Data.Map.Strict.

```
data Map k a  = Bin {-# UNPACK #-} !Size !k a !(Map k
a) !(Map k a)        -- spine strict !(Map k a)
              | Tip
```

value strictness is enforced only on insert in Data.Map.Strict
https://hackage-content.haskell.org/package/containers-0.8/docs/src/Data.Map.Strict.Internal.html#insert

```
insert :: Ord k => k -> a -> Map k a -> Map k a
insert = go
  where
    go :: Ord k => k -> a -> Map k a -> Map k a
    go !kx !x Tip = singleton kx
x                         -- value strict by !x
    go kx x (Bin sz ky y l r) =
        case compare kx ky of
            LT -> balanceL ky y (go kx x l) r
            GT -> balanceR ky y l (go kx x r)
            EQ -> Bin sz kx x l r
```

strictness of monads
[Discourse] Why shouldn't I make my monads value strict: https://discourse.haskell.org/t/why-shouldnt-i-make-my-monads-value-strict/8609
In general, a **monad is strict** if it is strict in its **computation**.
i.e.

if **k :: a -> m b** is strict then **_ >>= k :: m a -> mb** is strict

is also strict.
In other words, the bind **preserves the strictness** of the computation k.

```
import Data.Functor
import Data.Tuple

undefined >>= return @Solo `seq` ()            --
EXCEPTION strict
undefined >>= return @IO `seq` ()              -- ()
lazy
undefined >>= return @((->) ()) `seq` ()       -- ()
lazy
```

GOTCHA: But this can be more nuanced depending on the **type**.
e.g. Identity is actually lazy, but behaves close to strict due to it being a **newtype**.
i.e. strictness comes from the **data type**, not the bind.

```
newtype Identity a = Identity a
>>= :: Identity a -> (a -> Identity b) -> Identity b
m >>= k = k (runIdentity m)                    lazy due to
runIdentity

data Solo a = MkSolo a
>>= :: Solo a -> (a -> Solo b) -> Solo b       strict
MkSolo x >>= k = k x
```

```
-- Identity is lazy in computation
undefined >>= return @Identity `seq` ()            --
EXCEPTION due to newtype
undefined >>= return @Identity . Just `seq` ()     -- ()


-- Solo is strict in computation
undefined >>= return @Solo `seq` ()                --
EXCEPTION
undefined >>= return @Solo . Just `seq` ()         --
EXCEPTION
```

c) strictness in **pattern matching**
Bang pattern: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/strict.html

Pattern matching is **(shallow) strict** by **default**
e.g.

```
f :: a -> ...
f undefined = undefined
```

```
f :: (a, b) -> ...
f undefined = undefined

f :: Maybe a -> ...
f undefined = undefined
```

But patterns are only **shallow** strict by default
A **bang pattern (!)** is needed for **deeper** strictness.
e.g.

```
{- LANGUAGE BangPattern -}

f :: (a, a) -> [a]                    -- shallow strict
only
f (x, y) = [x, y]                     -- i.e. strict in
(,) but not the inner values
f undefined = undefined
f (undefined, undefined) = [undefined, undefined]


f :: (a, a) -> [a]                    -- deep strict in
first value
f (!x, y) = [x, y]
f !(!x, y) = [x, y]                   -- OK, but
meaningless
f undefined = undefined
f (undefined, 0) = undefined          -- strict in first
f (0, undefined) = [0, undefined]     -- non-strict in
second


f :: Maybe a -> [a]                   -- strict with
data type pattern
f (Just !a) = [a]
f Nothing = []
f undefined = undefined
f (Just undefined) = undefined
```

Conversely, patterns can be made **non-strict** with a **laziness flag ~**
e.g.

```
{- LANGUAGE BangPattern -}

f :: (a, a) -> [a]                -- all non-strict
f ~(x, y) = [x, y]
f v = [fst v, snd v]              -- equivalent

f ~(~x, y) = [x, y]               -- OK, but meaningless

f undefined = [undefined, undefined]
f (undefined, undefined) = [undefined, undefined]
```

Bottom
Haskell report: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-230003.1
[wiwinwlh] bottom: https://sdiehl.github.io/wiwinwlh/#bottoms
A bottom is a representation of an **invalid expression** in **non-strict** semantics.
Hence it can be used like any other value in expressions.
A bottom is **singular** i.e. can replace any type.

```
undefined :: Int          -- bottom as Int
undefined :: String       -- bottom as String
undefined :: () -> ()     -- bottom as function
```

In theory, a bottom is equivalent to a **non-terminating** expression

```
f = let x = x in x        -- non-terminating
```

Haskell has 2 **explicit** bottom functions:
a) **undefined**
b) **error** function

```
undefined                 -- undefined function
error "OOPS"              -- error function
```

There are many builtin ways that **induce** bottoms.
Such bottoms generate an **exception** at runtime to distinguish different sources.
Exception types: https://hackage-content.haskell.org/package/base/docs/Control-Exception.html

```
error "OOPS"              -- ErrorCall
1 `div` 0                 -- DivideByZero ::
ArithException
let (a, b) = undefined    -- PatternMatchFail
case 1 of                 -- patError
  0 -> ...
head []                   -- error "Prelude.head: empty
list"

data A = A { v::Int }     -- RecConError
A = A {}
```

==========
======

== Strictness Analysis ==

Strictness analysis

Demand analysis
Wiki: https://wiki.haskell.org/index.php?title=Performance/Strictness
Demand analysis paper: https://simon.peytonjones.org/theory-and-practice/

Compiler optimization by identifying **strict values** and evaluating it **immediately (call by value)** instead of **lazily (call by need)**.
i.e. NO **thunks**.
e.g. strict in z without strictness annotation.

```
f :: (Int, Int) -> ...
f (x, y) = if x == y then ...      -- compiler sees that
this is strict in x and y
                                   -- Computes x == y
immediately i.e. not lazy
f (!x, !y) = if x == y then ...    -- OK, but not needed
```

Worker-wrapper split
Demand: https://gitlab.haskell.org/ghc/ghc/-/wikis/commentary/compiler/demand
A form of **normalization** performed on a function:
- reduce **data type** argument to values
e.g.
  f :: (Int, Int) -> ...
  $wf :: Int -> Int

- use **unboxed values** where possible
e.g.
  f :: Int -> ...
  $wf :: #Int -> ...

e.g.

```
f :: (Int, Int) -> Int             -- wrapper
f v = case v of { (x, y) ->
  case x of { x' ->                -- unbox x :: Int ->
x' :: Int#
  case y of { y' ->                -- x = I# x'
     $wf x' y'
  }
  }
}

$wf :: Int# -> Int# -> Int         -- worker
$wf x y = I# (#+ x y)
```

e.g. worker wrapper split with factorial
script.hs

```
#!/usr/bin/env stack

module Main where

fact :: Int -> Int -> Int
fact r 0 = r
```

```
fact r n = fact (n * r) (n - 1)
fac :: Int -> Int
fac = fact 1

main = print $ fac 10000
```

Dump worker wrapper split

```
$ stack ghc -- -O2 script.hs  -ddump-worker-wrapper
-fforce-recomp
```

Output

```
...

Rec {
-- RHS size: {terms: 34, types: 12, coercions: 0, joins:
0/2}
$wfact_s2qD [InlPrag=[2], Occ=LoopBreaker]
  :: GHC.Prim.Int# -> GHC.Prim.Int# -> GHC.Prim.Int#
[LclId[StrictWorker([])], Arity=2, Str=<L><SL>]
$wfact_s2qD
                     -- Worker
  = \ (ww_s2qv :: GHC.Prim.Int#)
      (ww_s2qz [Dmd=SL] :: GHC.Prim.Int#) ->
      case let {
             r_a1ex [Dmd=S!P(L)] :: Int
             [LclId]
             r_a1ex = GHC.Types.I# ww_s2qv } in
           let {
             ds_d1za [Dmd=S!P(SL)] :: Int
             [LclId]
             ds_d1za = GHC.Types.I# ww_s2qz } in
           case ds_d1za of { GHC.Types.I# ds_d1zb
[Dmd=SL] ->
           case ds_d1zb of ds_X1 {
             __DEFAULT ->
               fact
                 (case r_a1ex of { GHC.Types.I# y_a1B0
->
                  GHC.Types.I# (GHC.Prim.*# ds_X1
y_a1B0)
                  })
                 (GHC.Types.I# (GHC.Prim.-# ds_X1 1#));
             0# -> r_a1ex
           }
           }
      of ww_s2qB
      { __DEFAULT ->
      case ww_s2qB of wild_00 { GHC.Types.I# ww_s2qH ->
ww_s2qH }
      }
```

```
-- RHS size: {terms: 15, types: 7, coercions: 0, joins:
0/0}                  -- Wrapper
fact [InlPrag=[2], Dmd=LC(L,C(1,!P(L)))] :: Int -> Int
-> Int
[LclId,
 Arity=2,
 Str=<S!P(L)><S!P(SL)>,
 Cpr=1,
 Unf=Unf{Src=StableSystem, TopLvl=True,
         Value=True, ConLike=True, WorkFree=True,
Expandable=True,

Guidance=ALWAYS_IF(arity=2,unsat_ok=True,boring_ok=False
)
         Tmpl= \ (r_s2qt [Occ=Once1!, Dmd=S!P(L)] ::
Int)
                 (ds_s2qx [Occ=Once1!, Dmd=S!P(SL)] ::
Int) ->
                 case case r_s2qt of { GHC.Types.I#
ww_s2qv [Occ=Once1] ->
                      case ds_s2qx of { GHC.Types.I#
ww_s2qz [Occ=Once1, Dmd=SL] ->
                      $wfact_s2qD ww_s2qv ww_s2qz
                      }
                      }
                 of ww_s2qH [Occ=Once1]
                 { __DEFAULT ->
                 GHC.Types.I# ww_s2qH
                 }}]
fact
  = \ (r_s2qt [Dmd=S!P(L)] :: Int) (ds_s2qx [Dmd=S!
P(SL)] :: Int) ->
      case case r_s2qt of wild_00 { GHC.Types.I# ww_s2qv
->
           case ds_s2qx of wild_00 { GHC.Types.I#
ww_s2qz [Dmd=SL] ->
           $wfact_s2qD ww_s2qv ww_s2qz
           }
           }
      of ww_s2qH
      { __DEFAULT ->
      GHC.Types.I# ww_s2qH
      }
end Rec }
```

equivalent

```
fact
    -- Wrapper
  = \ (r :: Int) (n :: Int) ->
      case case r of { I# r' ->
           case n of { I# n' ->
           $wfact_s2qD r' n'
           }
```

```
      }

$wfact :: Int# -> Int# -> Int
    -- Worker
$wfact
  = \ (r' :: Int#)
      (n' :: Int#) ->
      let
        r, n :: Int
        r = I# r'
        n = I# n'
      in
         case r of { I# r'' ->
            0# -> r
            _   -> fact
                     (case n of { Int# n'' -> I# (*# r''
n'') })
                     (I# (-# n'' 1#));
           }
       }
```

==========

== Lazy evaluation (evaluation strategy) ==

Wiki: https://en.wikipedia.org/wiki/Lazy_evaluation
Haskell Wiki: https://wiki.haskell.org/Lazy_evaluation

Lazy (call-by-need) vs Strict (call-by-value) evaluation
Strict evaluation: https://www.haskell.org/onlinereport/haskell2010/haskellch6.html#x13-1260006.2
[wiwinwlh] laziness: https://sdiehl.github.io/wiwinwlh/#laziness
Lazy means that by default, evaluation only occurs by **need**
i.e. in IO

```
y = f x     :: [a]        -- f <thunk> i.e. thunk not
evaluated

print (f x) :: IO ()      -- evaluate
```

In practice, this can have negative effects
e.g. see Lazy evaluation and **performance**.

Thus Haskell has 2 ways to make evaluation more strict:
A) **force** evaluation:
i.e. with **seq** and **bang (!) patterns**

b) switch to **call-by-value (aka strict evaluation)** instead of **call-by-need** (lazy evaluation):
Using **strictness analysis**.

==========
======

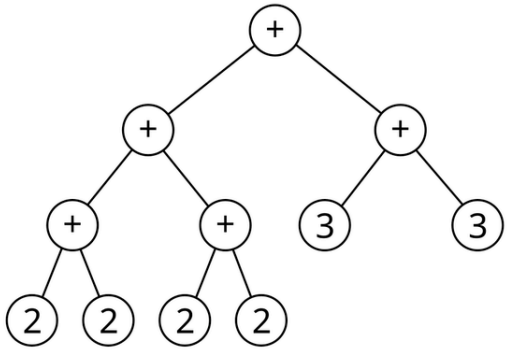
== Graph reduction (operation semantics) ==

Wiki: https://en.wikipedia.org/wiki/Graph_reduction
Book: https://en.wikibooks.org/wiki/Haskell/Graph_reduction

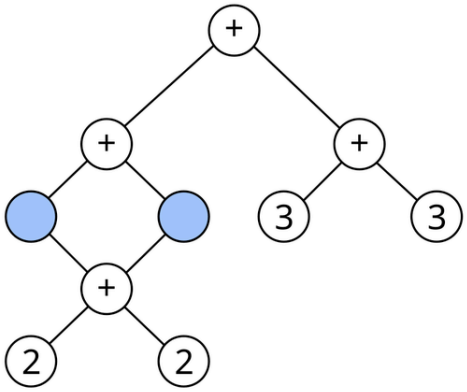
Graph reduction
Graph reduction is a **reduction strategy i.e. operation semantics** for **lazy evaluation** in Haskell.
Graph reduction is **outermost (breadth first) + sharing** reduction.

Graph because **sharing** is expressed as a **cycle (undirected cycle i.e. cyclic edges but not in the same direction)** in the reduction.
Hence the original **reduction tree** becomes a **reduction graph (directed acylic graph)**.

e.g. reduction of (2+2)+(2+2)+(3+3)
Original reduction tree



Reduction graph with sharing
i.e. 2+2 is **shared** once



Normal form
Weak head normal form (WHNF)
Wiki: https://wiki.haskell.org/Weak_head_normal_form

Wiki - Graph reduction: https://en.wikibooks.org/wiki/Haskell/Graph_reduction#Weak_Head_Normal_Form
What is weak head normal form?: https://stackoverflow.com/questions/6872898/what-is-weak-head-normal-form
[wiwinwlh] strictness: https://sdiehl.github.io/wiwinwlh/#strictness
Weak head normal form is a **normal form** for **outermost evaluation.**
i.e. normal form for **non-strict** evaluation
Outermost evaluation evaluates the **outer layer** first, but this requires a definition of what makes an outer layer.
i.e. a definition of the **normal form** of an **outer layer**.
This definition is the weak head normal form.

normal form expression fully evaluated down to **every level**

weak head normal form (WHNF) expression fully evaluated to **outermost level**

In general WHNF means the **outermost data type**, including lambdas and builtins.
To be precise:
a) a **data constructor**
e.g.
Just 1
Just undefined

NOTE: **newtype** is equivalent to the inner type
e.g.
Identity undefined same as undefined
undefined

b) a **partially applied** builtin function
e.g.
sqrt
(+1)
undefined

c) lambda, including builtin values
1
()
\x -> ...

e.g. test if WHNF using seq

```
Just undefined            `seq` ()         -- OK WHNF =
Just _
(undefined :: Maybe a)    `seq` ()         -- EXCEPTION
(undefined:undefined)     `seq` ()         -- OK WHNF =
_:_ for lists
```

***) Basic evaluation rules**
[SPJ] Implementation of functional programming languages: https://simon.peytonjones.org/slpj-book-1987/
Basic evaluation applies the rules of **lambda calculus** (see lambda calculus concept notes).
i.e. substitution, α-reduction, β-reduction, η-reduction

	Description	Example
α-reduction (alpha)	**replace variable** name i.e. **equivalence** by variable name	f = \y -> y + 1 => f = g = \x -> x + 1 g= \x -> x + 1
β-reduction (beta)	function **application**	(\x -> x+1) 10 => (10 + 1)
η-reduction (eta)	**remove** redundant variable	\x -> f x => f

Function rewrite
There are no **function declarations** in lambda calculus.
Thus, in order to apply lambda calculus, **functions** need to be **rewritten** as **lambdas** with a **single argument**.
This requires:
- rewrite **function calls** with **lambda application**
- rewrite **argument pattern matches** with **case**
- rewrite **multiple arguments** with **currying**
i.e.

original		rewrite
pattern match	f :: Maybe a -> ... f (Just a) = ... f Nothing = ...	**case expression** f x = case x of Just a -> ... Nothing -> ...
function declaration	f :: a -> b -> ... f x = ...	**lambda** \x, y -> ...
multiple arguments	f :: a -> b -> f x y = ..	**currying** \x -> \y ->

e.g.
rewrite

```
f :: Maybe a -> b -> ...
f (Just x) w = <e1>
f Nothing w = <e2>

-- Step 1: replace pattern match with case
f v w = case v of
  Just x -> <e1>
  Nothing -> <e2>

-- Step 2: replace function with lambda
\v, w -> case v of        -- lambda + case
  Just x -> <e1>
```

```
  Nothing -> <e2>

-- Step 3: curry
\v ->
  \w -> case v of
    Just x -> <e1>
    Nothing -> <e2>
```

Reduce a function call

```
f (Just 1) ()
 => (\v ->                    -- function rewrite
      \w -> case v of
        Just x -> <e1>
        Nothing -> <e2>
    ) (Just 1) ()
 => case (Just 1) of          -- beta reduce
        Just x -> <e1>
        Nothing -> <e2>
 => <e1>
```

***) Evaluation order**

The basic rules of expression evaluation are defined by **lambda calculus**.
However, lambda calculus does not specify the order of evaluation.
i.e. whether it is **outermost (breadth first)** or **innermost (depth first)**.

Rule a) **expression order (WHNF)**
Expressions are evaluated
a) **left** most to right
b) **outermost i.e. weak head normal form**

Outermost means reduce only upto **weak head** at each level

e.g. outermost reduction via WHNF

```
x = ([1,2,3,...], [1...])
head (fst x)

head (fst x)
  => (\v -> case v of (w:_) -> w)          lambda
rewrite
     (\v -> case v of (w,_) -> w)
     x

  => (\v -> case v of (w:_) -> w)          beta reduce:
apply x on lambda
     (case x of (w,_) -> w)

  => (\v -> case v of (w:_) -> w)
     (case (<e1>, <e2>) of (w,_) -> w)     reduce x to
WHNF of pair (_,_)

  => (\v -> case v of (w:_) -> w)          reduce
```

```
pattern match
     <e1>                                    NOTE: <e2> is
NOT evaluated

  => case <e1> of (w:_) -> w                 beta reduce:
apply <e1> on lambda

  => case (<e3>:<e4>) of (w:_) -> w          reduce <e1>
to WHNF of list (_:_)

  => <e3>                                    reduce
pattern match
  => 1                                       reduce to
WHNF of Int
                                             NOTE: <e4> is
not evaluated



-- Reduction
x
  => (<e1>,<e2>)      reduce to WHNF of pair (_,_)
                      NOTE: <e2> NOT evaluated
  => <e1>             beta reduce by

  => 1:<e3>           reduce to WHNF of list _:_
                      NOTE: <e3> NOT evaluated
  => 1                WHNF of Int
```

Rule b) **pattern match order**
Patterns are matched
1) **top** first i.e. top to bottom
2) **left** first i.e. left to right

Order makes a difference
e.g.

```
or :: Bool -> Bool -> Bool
or True x = True
or _ x = x

v = not x

-- Left to right + top to bottom
or (1==1) v
  => or True v                    -- OK True
  => True

-- Bottom to top
or (1==1) v
  => or _ (not v)                 -- BAD. True pattern
ignored
  => or _ (not (not v))           -- never terminates
  => ...
```

```
-- Right to left
or (1==1) v
  => or (1==1) (not v)
  => or (1==1) (not (not v))      -- BAD. v evaluated
before condition
  => ...                          -- never terminates
```

==========

==OLD==

bind	result	example
m _|_ >> m b	_|_	?
	m _|_	IO a
	m b	Identity a, ((), a), Maybe a, Solo a, () -> a
_|_ >> m b	_|_	((), a), Maybe a, Solo a
	m _|_	IO a
	m b	Identity a, () -> a
m b >> m _|_	_|_	Identity a
	m _|_	IO a, () -> a, ((), a), Solo a
	m b	Maybe a
m b >> _|_	_|_	Identity a, ((), a), Solo a
	m _|_	IO a, () -> a
	m b	Maybe a

```
-- Identity (new type)
-- data type:  strict
-- monad bind: lazy in first, strict in second
Identity undefined            -- undefined
strict data type
undefined >> Identity ()      -- Identity ()
lazy in first
Identity () >> undefined      -- undefined
strict in second


-- Solo (data type)
-- data type:  lazy
-- monad bind: strict in both
MkSolo undefined                  -- MkSolo undefined
lazy data type
undefined >> MkSolo ()            -- undefined
strict in first
MkSolo undefined >> MkSolo ()     -- MkSolo ()
MkSolo () >> undefined            -- undefined
```

```
strict in second
MkSolo () >> MkSolo ()            -- MkSolo undefined


-- Maybe
-- data type:  lazy
-- monad bind: strict in first, lazy in second
Just undefined                -- Just undefined    lazy
undefined >> Nothing          -- undefined
strict in first
undefined >> Just ()          -- undefined
Just undefined >> Nothing     -- Nothing
Just undefined >> Just ()     -- Just ()
Nothing >> undefined          -- Nothing           lazy
in second
Just () >> undefined          -- undefined
Nothing >> Just undefined     -- Nothing
Just () >> Just undefined     -- Just undefined


-- IO
-- data type: lazy
return undefined              :: IO ()     -- IO
undefined
undefined >> return ()        :: IO ()     -- IO
undefined
return undefined >> return () :: IO ()     -- IO
undefined
return () >> undefined        :: IO ()     -- IO
undefined
return () >> return undefined :: IO ()     -- IO
undefined

-- () -> a
-- data type: lazy
undefined >> const ()             -- const ()
return undefined >> const ()      -- const ()
const () >> undefined             -- const undefined
const () >> return undefined      -- const undefined
```

Hence Haskell has a way to **strict evaluation aka call by value**.

A **complete force evaluation** to **normal form** also has problems:
1) **expensive**
2) defeats the point of **laziness**

Hence Haskell implements a middle ground **shallow force evaluation** to **weak head normal form**.

	default	**seq**	**deepseq**
evaluation	none	WHNF	NF

evaluation depth	none	shallow	deep
evaluation cost	none	cheap	expensive
accumulated thunks	high	medium	none

seq, deepseq
seq for **shallow evaluation** to WHNF
bang (!) syntax for syntactic sugar of seq

```
-- seq
x `seq` ...          -- evaluate x
                     -- same effect as e.g. print x,
without the effect

-- Bang pattern
f !x = ...
f x = `x` seq ...    --
f (!x, !y) = ...
let !x = ...         -- in let
f $! x               -- evaluate x before calling f
                     -- i.e. f $! x = let !v = x in f v
```

deepseq for **deep evaluation** to NF
e.g. https://hackage.haskell.org/package/deepseq

```
x = Just (...)

1 `seq` ()                        -- ()
undefined `seq` ()                -- undefined
Just undefined `seq` ()           -- ()
Just undefined `deepseq` ()       -- undefined
```

```
This
is
code
Monaco 14
```